

Parallel & Distributed Computing

FINAL-TERM COMPREHENSIVE HANDOUT

GPU & CUDA • Advanced OpenMP/MPI • Parallel Algorithms • Consensus • Big Data
• Cloud

BS Computer Science — Cumulative (Full Semester) — July 2026

 **Final-Term Exam Coverage (Cumulative):** This handout is a **single, all-in-one study document** for the final examination. **Part I** is a condensed recap of the midterm material (fundamentals, performance, synchronization, distributed-systems basics) delivered as quick-reference tables and formula boxes. **Part II** covers all new final-term topics in depth: advanced shared-memory programming (OpenMP tasks, Pthreads), GPU/CUDA computing, advanced MPI, parallel algorithms, distributed coordination & mutual exclusion, consensus (2PC/3PC/Paxos/Raft/PBFT), distributed transactions, MapReduce & Spark, distributed file systems, cloud/virtualization/containers, and performance engineering. Focus on **formulae, definitions, algorithms, and comparison tables**.

Contents

I	Part I — Midterm Recap (Condensed)	4
1	Fundamentals at a Glance	4
1.1	Parallel vs Distributed — Summary	4
1.2	Flynn's Taxonomy & Parallelism Types	4
2	Performance Metrics — Recap	5
3	Design, Synchronization & Distributed Basics — Recap	5
3.1	PCAM Design Methodology (Foster)	5
3.2	Synchronization Primitives	5
3.3	Distributed Systems Basics	6
II	Part II — Final-Term Core Topics	7
4	Advanced Shared-Memory Programming	7
4.1	OpenMP Beyond the Parallel For	7
4.1.1	Loop Scheduling	7

4.1.2	Tasks and Sections	7
4.1.3	Data-Sharing & Reductions	8
4.2	POSIX Threads (Pthreads)	8
4.3	Thread Pools & Work Stealing	9
5	GPU Computing & CUDA	9
5.1	The SIMT Execution Model	10
5.2	Thread Hierarchy	10
5.3	Memory Hierarchy	10
5.4	Performance Levers	11
6	Advanced Message Passing (MPI)	11
6.1	Blocking, Non-Blocking & Communication Modes	11
6.2	Collective Operations (Deep Dive)	12
6.3	Datatypes, Topologies & One-Sided	12
7	Parallel Algorithms	13
7.1	Parallel Reduction & Prefix Sum (Scan)	13
7.2	Parallel Sorting	13
7.3	Parallel Matrix Multiplication — Cannon’s Algorithm	13
8	Distributed Coordination & Mutual Exclusion	14
8.1	Distributed Mutual Exclusion	14
8.2	Leader Election	14
8.3	Distributed Deadlock	15
9	Distributed Consensus & Agreement	15
9.1	Atomic Commit: Two-Phase & Three-Phase Commit	15
9.2	Paxos	15
9.3	Raft — Understandable Consensus	16
9.4	Byzantine Fault Tolerance (PBFT)	16
10	Distributed Transactions & Data	16
10.1	Concurrency Control & Quorums	17
10.2	Replication Strategies & NoSQL Models	17
11	MapReduce & Big-Data Frameworks	17
11.1	Hadoop Ecosystem	18
11.2	Apache Spark	18
12	Distributed File Systems & Storage	19
13	Cloud, Virtualization & Containers	19
14	Performance Engineering & Analysis	20
14.1	Diagnosing and Improving Performance	20

15 Emerging & Applied Topics	21
Final-Term Formula & Fact Sheet	22

Part I — Midterm Recap (Condensed)

💡 How to use Part I

Part I is a fast refresher, not a re-teaching. Each block collapses a midterm chapter into its exam-critical definitions, formulae, and comparison tables. If any row is unfamiliar, revisit the midterm handout (`PDC_Handout.pdf`) before moving to Part II.

1 Fundamentals at a Glance

📖 Core Definitions

Parallel Computing: many calculations performed **simultaneously**; one problem split across multiple processors, usually tightly coupled.

Distributed Computing: computation across multiple **networked, autonomous** computers that coordinate by **message passing** toward a common goal.

Concurrency vs Parallelism: concurrency = *dealing with* many things at once (composition/structure); parallelism = *doing* many things at once (simultaneous execution).

1.1 Parallel vs Distributed — Summary

Aspect	Parallel	Distributed
Memory	Shared or local, one site	Private per node, geographically spread
Communication	Shared variables / bus	Message passing over network
Clock	Often global/tight	No global clock; loose coupling
Failures	Rare, affect whole system	Frequent, partial failures normal
Primary goal	Speed (performance)	Scalability & fault tolerance
Examples	Multi-core, GPU, OpenMP, CUDA	Clusters, Cloud, MPI, Hadoop

1.2 Flynn's Taxonomy & Parallelism Types

Class	Meaning	Example
SISD	Single instruction, single data (sequential)	Classic von Neumann CPU
SIMD	Single instruction, multiple data	GPU, AVX/SSE vector units
MISD	Multiple instruction, single data (rare)	Fault-tolerant pipelines
MIMD	Multiple instruction, multiple data	Multi-core, clusters

- **Data parallelism:** same operation over different data partitions (SIMD-style).
- **Task parallelism:** different operations run concurrently (MIMD-style).

- **Pipeline parallelism:** staged assembly line; stage i processes item n while stage $i-1$ processes $n+1$.
- **Memory models:** **Shared** (UMA/NUMA/ccNUMA), **Distributed** (message passing), **Hybrid** (multi-core nodes + interconnect, e.g. MPI+OpenMP).

2 Performance Metrics — Recap

The Five Formulae You Must Know

Speedup: $S(p) = \frac{T_s}{T_p}$

Efficiency: $E(p) = \frac{S(p)}{p} = \frac{T_s}{p T_p}$

Amdahl: $S(p) = \frac{1}{(1-f) + \frac{f}{p}}$

Amdahl limit: $S_{\max} = \frac{1}{1-f}$

Gustafson: $S(p) = f p + (1-f)$

T_s = serial time, T_p = time on p processors, f = parallel fraction.

Amdahl vs Gustafson — the exam trap

Amdahl fixes the *problem size* $\Rightarrow$ speedup is capped by the serial fraction (pessimistic, models **strong scaling**). **Gustafson** lets the *problem grow* with $p \Rightarrow$ near-linear speedup is achievable (optimistic, models **weak scaling**). Same hardware, different assumption about workload.

Term	Definition	Note
Linear speedup	$S = p$	Ideal
Superlinear	$S > p$	Rare; cache/memory effects
Strong scaling	Fixed problem, more p	Bounded by Amdahl
Weak scaling	Problem grows with p	Governed by Gustafson
Scale up (vertical)	Bigger machine	Limited by hardware
Scale out (horizontal)	More machines	Distributed systems path

3 Design, Synchronization & Distributed Basics — Recap

3.1 PCAM Design Methodology (Foster)

Partitioning $\rightarrow$ Communication $\rightarrow$ Agglomeration $\rightarrow$ Mapping. Start with the finest-grain tasks (domain or functional decomposition), identify data exchange, combine tasks to cut overhead, then place tasks on processors. **Load balancing** may be *static* (fixed upfront) or *dynamic* (runtime, e.g. work stealing / task queues).

3.2 Synchronization Primitives

Primitive	Purpose
Mutex	Binary lock; one holder at a time.

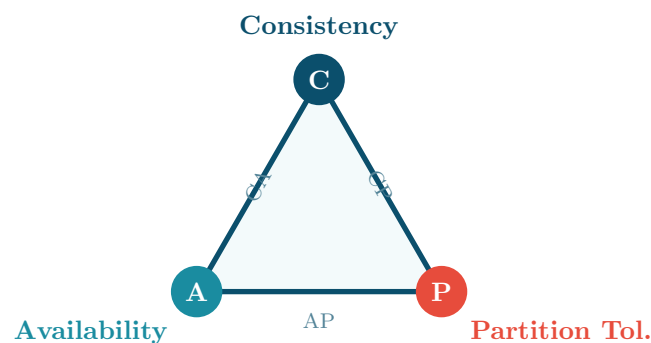
Primitive	Purpose
Semaphore	Integer with atomic P (wait/down) / V (signal/up); binary $\approx$ mutex, counting guards n resources.
Monitor	High-level construct; auto mutual exclusion + condition variables (Java <code>synchronized</code>).
Barrier	All threads must arrive before any proceeds; phase separation.

⚠ Critical Section & Deadlock

A **critical section** needs **Mutual Exclusion**, **Progress**, and **Bounded Waiting**. A **deadlock** requires all four Coffman conditions simultaneously: **Mutual Exclusion**, **Hold & Wait**, **No Preemption**, **Circular Wait**. Break any one to prevent it (avoidance = Banker's Algorithm; also detect & recover). Related pathologies: **livelock** (state changes, no progress) and **starvation** (perpetual denial).

3.3 Distributed Systems Basics

- **Goals:** transparency, openness, scalability, fault tolerance, performance. Transparency kinds: access, location, migration, replication, concurrency, failure.
- **CAP Theorem:** pick at most 2 of {Consistency, Availability, Partition tolerance}. Since partitions are inevitable, real systems choose **CP** (e.g. HBase, Zookeeper) or **AP** (e.g. Cassandra, DynamoDB).
- **Consistency models (strong $\rightarrow$ weak):** strong, sequential, causal, eventual; plus client guarantees read-your-writes, monotonic reads.
- **Logical clocks:** **Lamport** ($a \rightarrow b \Rightarrow L(a) < L(b)$, not iff) and **Vector clocks** ($V(a) < V(b) \Leftrightarrow a \rightarrow b$, captures causality exactly).
- **Fault tolerance:** crash, omission, timing, response, **Byzantine** (arbitrary). BFT consensus needs $n \geq 3f + 1$.



Part II — Final-Term Core Topics

4 Advanced Shared-Memory Programming

4.1 OpenMP Beyond the Parallel For

💡 The Fork–Join Model

An OpenMP program runs as a single **master thread** that **forks** a team of threads at a **parallel region** and **joins** them at its end. Work-sharing constructs (**for**, **sections**, **single**, **task**) divide work among the team.

4.1.1 Loop Scheduling

The `schedule` clause controls how loop iterations map to threads — crucial when iterations have **unequal cost**.

Schedule	Behaviour	Best for
<code>static, c</code>	Iterations split into fixed chunks of size <code>c</code> , assigned round-robin at compile/entry time. Lowest overhead.	Uniform iteration cost
<code>dynamic, c</code>	Threads grab chunks of size <code>c</code> from a shared queue as they finish. Balances load, higher overhead.	Irregular workloads
<code>guided, c</code>	Like dynamic but chunk size shrinks over time (large first, then small).	Unknown/irregular, tail balance
<code>runtime</code>	Decided by <code>OMP_SCHEDULE</code> environment variable.	Tuning without recompiling
<code>auto</code>	Compiler/runtime chooses.	Let the system decide

4.1.2 Tasks and Sections

🔗 OpenMP Tasks — Irregular / Recursive Parallelism

```
// Parallel recursive Fibonacci using tasks
int fib(int n) {
    if (n < 2) return n;
    int x, y;
    #pragma omp task shared(x)           // spawn independent task
    x = fib(n - 1);
    #pragma omp task shared(y)
    y = fib(n - 2);
    #pragma omp taskwait                 // wait for both child tasks
    return x + y;
}
// Called once inside: #pragma omp parallel { #pragma omp single {
//     fib(n); } }
```

- **task** — creates a unit of deferred work placed in a pool; ideal for **unbounded/recursive** structures (trees, graphs, linked lists) where iteration count is unknown.
- **taskwait** — suspends the current task until its child tasks complete.
- **sections** / **section** — assign distinct code blocks to different threads (functional/task parallelism).
- **single** / **master** — only one thread (or the master) executes the enclosed block.

4.1.3 Data-Sharing & Reductions

Clause	Meaning
shared(x)	One instance of x shared by all threads.
private(x)	Each thread gets an uninitialised private copy.
firstprivate(x)	Private copy initialised to the value before the region.
lastprivate(x)	Value from the logically last iteration copied out.
reduction(op:x)	Private copies combined with op (+, *, max , min , &&) at the end.

⚠ False Sharing — a Silent Performance Killer

When two threads repeatedly write to **different variables that share the same cache line**, the cache-coherence protocol keeps invalidating that line, forcing it to bounce between cores. The code is correct but slow. **Fix:** pad/align data so per-thread variables occupy separate cache lines, or use thread-local accumulation and combine at the end.

4.2 POSIX Threads (Pthreads)

💡 Pthreads — Explicit Low-Level Threading

Where OpenMP is directive-based and implicit, **Pthreads** is a library giving you **explicit** control of thread creation, joining, and synchronisation — more boilerplate, more control.

🔗 Pthreads: create, join, mutex

```
#include <pthread.h>
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long counter = 0;

void *worker(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);    // enter critical section
        counter++;
        pthread_mutex_unlock(&lock);  // leave critical section
    }
    return NULL;
}

int main() {
    pthread_t t[4];
    for (int i = 0; i < 4; i++) pthread_create(&t[i], NULL, worker,
        , NULL);
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL); // wait
        for all
```

}

Pthreads Call	Role
<code>pthread_create</code>	Start a new thread running a given function.
<code>pthread_join</code>	Block until a target thread finishes (collect it).
<code>pthread_mutex_lock/unlock</code>	Acquire / release a mutual-exclusion lock.
<code>pthread_cond_wait/signal</code>	Condition variables: sleep until a predicate holds / wake a waiter.
<code>pthread_barrier_wait</code>	Barrier synchronisation across threads.

4.3 Thread Pools & Work Stealing

- **Thread pool:** a fixed set of worker threads pulling tasks from a shared queue — avoids the cost of creating/destroying threads per task.
- **Work stealing:** each worker owns a double-ended queue (deque); it pushes/pops its own tasks from one end, and **idle workers steal** from the other end of a busy worker's deque. Used in Java ForkJoinPool, Intel TBB, Cilk. Excellent dynamic load balance with low contention.

Feature	OpenMP	Pthreads
Abstraction	High (directives)	Low (explicit library)
Effort	Incremental, small diff	Verbose, manual
Control	Coarser	Fine-grained
Load balancing	Built-in schedules	You implement it
Best when	Loop/data parallelism	Custom thread topologies

5 GPU Computing & CUDA

💡 Why GPUs?

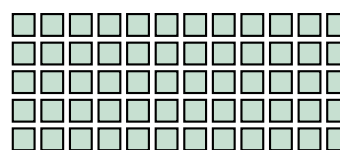
A CPU has a few powerful cores optimised for **latency** (fast single-thread, big caches, branch prediction). A GPU has **thousands** of simple cores optimised for **throughput**: run enormous numbers of threads so that memory-stalled threads are hidden by others that are ready. Ideal for **data-parallel**, arithmetic-heavy, regular workloads (matrices, images, neural networks).

CPU: few large cores



latency-optimised

GPU: thousands of small cores



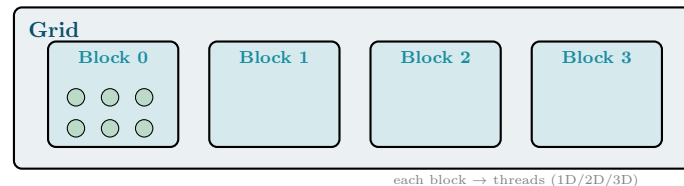
throughput-optimised

5.1 The SIMT Execution Model

SIMT — Single Instruction, Multiple Threads

CUDA threads are grouped into **warps** of 32 that execute the **same instruction in lockstep** on different data. If threads in a warp take different branch paths (**warp divergence**), the paths are *serialised*, hurting performance. Keep threads in a warp on the same control-flow path.

5.2 Thread Hierarchy



- **Thread** — runs the kernel; identified by `threadIdx`.
- **Block** — group of threads that can cooperate via **shared memory** and `__syncthreads()`; identified by `blockIdx`, sized by `blockDim`.
- **Grid** — all blocks launched for one kernel; sized by `gridDim`.
- Global index: `int i = blockIdx.x * blockDim.x + threadIdx.x;`

5.3 Memory Hierarchy

Memory	Scope	Speed	Notes
Registers	Per thread	Fastest	Automatic variables
Shared memory	Per block	Very fast	On-chip; programmer-managed cache; use for data reuse
Global memory	Whole grid	Slow (high latency)	Main DRAM; coalesce accesses!
Constant memory	Read-only, all	Fast (cached)	Broadcast same value to warp
Local memory	Per thread	Slow	Register spills (actually in DRAM)

CUDA Kernel — Vector Addition

```
__global__ void vecAdd(const float *A, const float *B, float *C,
    int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x; // global
    // thread id
    if (i < n) C[i] = A[i] + B[i]; // guard
    // against overrun
}
int main() {
    float *dA, *dB, *dC;
    cudaMalloc(&dA, n*sizeof(float)); /* dB, dC similarly */
    cudaMemcpy(dA, hA, n*sizeof(float), cudaMemcpyHostToDevice);
    // H->D
```

```

int threads = 256, blocks = (n + threads - 1) / threads;
    // ceil div
vecAdd<<<blocks, threads>>>(dA, dB, dC, n);
    // launch
cudaMemcpy(hC, dC, n*sizeof(float), cudaMemcpyDeviceToHost);
    // D->H
cudaFree(dA); /* ... */
}

```

5.4 Performance Levers

- **Memory coalescing:** consecutive threads should access consecutive global-memory addresses so the hardware merges them into one wide transaction.
- **Occupancy:** ratio of active warps to the maximum a Streaming Multiprocessor (SM) supports. Higher occupancy hides latency; limited by registers/shared-memory per block.
- **Host–device transfer:** the PCIe bus is a bottleneck — minimise copies, keep data resident on the GPU, and overlap transfer with compute using **streams**.
- **Avoid divergence:** branch uniformly within a warp; avoid `if` that splits a warp.

⚠ CUDA vs OpenCL vs OpenMP-offload

CUDA = NVIDIA-only, mature, best tooling. **OpenCL** = open, portable across GPUs/CPU/FPGAs, more verbose. **OpenMP target/OpenACC** = directive-based GPU offloading, easiest to adopt. Exam angle: CUDA gives peak control/performance at the cost of portability.

6 Advanced Message Passing (MPI)

💡 Recap: MPI Essentials

Rank = process id $0..N-1$; **communicator** = a process group (`MPI_COMM_WORLD` = all). Point-to-point: `MPI_Send`/`MPI_Recv`. Collectives involve every process in a communicator. Final-term goes *deeper*: communication modes, datatypes, topologies, one-sided, and hybrid models.

6.1 Blocking, Non-Blocking & Communication Modes

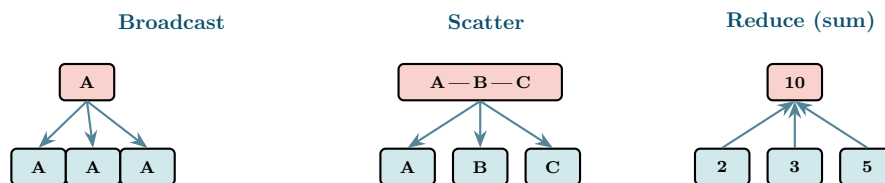
Call	Semantics	Use / Risk
<code>MPI_Send</code> (standard)	May buffer or block until receiver ready	Simple; deadlock risk if both send first
<code>MPI_Ssend</code> (synchronous)	Returns only when receive has started	Guarantees rendezvous
<code>MPI_Bsend</code> (buffered)	Copies to user buffer, returns immediately	Needs <code>MPI_Buffer_attach</code>
<code>MPI_Isend/Irecv</code>	Non-blocking; returns a request handle	Overlap comm. with compute; must <code>MPI_Wait/Test</code>

⚠ Classic MPI Deadlock

If every process calls a blocking `MPI_Send` to its neighbour *before* any `MPI_Recv`, and buffers are full, all processes wait forever. **Fixes:** reorder send/recv (odd-even), use `MPI_Sendrecv`, or use non-blocking `MPI_Isend`/`MPI_Irecv` then `MPI_Waitall`.

6.2 Collective Operations (Deep Dive)

Pattern	MPI Call	Effect
Barrier	<code>MPI_Barrier</code>	All ranks synchronise.
Broadcast	<code>MPI_Bcast</code>	Root's data copied to all ranks.
Scatter	<code>MPI_Scatter</code>	Root splits an array, one chunk per rank.
Gather	<code>MPI_Gather</code>	Root collects one chunk from each rank.
Allgather	<code>MPI_Allgather</code>	Gather with result delivered to all ranks.
Reduce	<code>MPI_Reduce</code>	Combine (sum/max/...) into root.
Allreduce	<code>MPI_Allreduce</code>	Reduce with result at all ranks (common in ML).
Reduce-scatter	<code>MPI_Reduce_scatter</code>	Reduce then scatter the result.
All-to-all	<code>MPI_Alltoall</code>	Every rank sends distinct data to every rank (transpose).
Scan	<code>MPI_Scan</code>	Prefix reduction across ranks.



6.3 Datatypes, Topologies & One-Sided

- **Derived datatypes:** describe non-contiguous data (`MPI_Type_vector`, `_contiguous`, `_struct`) so you can send a matrix column or a struct in one call.
- **Virtual topologies:** impose a logical shape (`MPI_Cart_create` for grids/tori) that maps neighbours cleanly and lets the runtime optimise placement.
- **One-sided (RMA):** `MPI_Put`/`MPI_Get`/`MPI_Accumulate` on a **window** of exposed memory — the origin drives the transfer without the target actively receiving. Reduces synchronisation.
- **Communicators:** `MPI_Comm_split` partitions ranks into sub-groups for scoped collectives.

💡 Hybrid MPI + OpenMP

Modern clusters are hierarchical: many multi-core nodes. The **hybrid model** uses **MPI between nodes** (distributed memory) and **OpenMP within a node** (shared memory). Benefits: fewer MPI ranks and messages, less memory replication, better use of intra-node bandwidth. This mirrors the **hybrid architecture** from Part I.

7 Parallel Algorithms

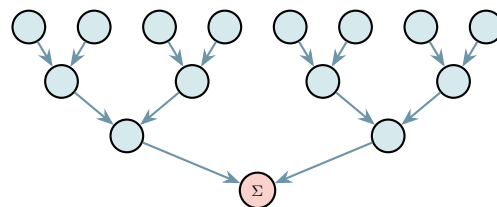
Work, Span & Cost

For a parallel computation: **Work** W = total operations (time on 1 processor); **Span** (depth) D = length of the longest dependency chain (time with ∞ processors). Then $T_p \geq \max(W/p, D)$ and parallelism = W/D . An algorithm is **cost-optimal** if $p \cdot T_p = \Theta(W_{\text{best serial}})$.

7.1 Parallel Reduction & Prefix Sum (Scan)

- **Reduction:** combine n values (sum/max) via a balanced tree in $O(\log n)$ span, $O(n)$ work.
- **Prefix sum (scan):** output $y_i = x_0 \oplus x_1 \oplus \dots \oplus x_i$. The **Hillis–Steele** scan runs in $O(\log n)$ steps ($O(n \log n)$ work); the work-efficient **Blelloch** scan uses up-sweep + down-sweep in $O(n)$ work, $O(\log n)$ span. Scan underpins stream compaction, sorting, and histograms.

Tree Reduction: 8 values in $\log_2 8 = 3$ steps



7.2 Parallel Sorting

Algorithm	Idea	Complexity / Notes
Odd–Even Transposition	Neighbour compare-exchange over odd/even phases	$O(n)$ phases; simple on arrays/rings
Bitonic Sort	Sort bitonic sequences via a fixed comparator network	$O(\log^2 n)$ depth; data-independent, great for GPUs/SIMD
Parallel Merge Sort	Divide, sort in parallel, parallel merge	$O(\log^2 n)$ span with parallel merge
Sample Sort	Pick splitters, bucket, sort buckets in parallel	Scales to clusters; near work-optimal

7.3 Parallel Matrix Multiplication — Cannon’s Algorithm

💡 Cannon’s Algorithm

Multiply $A \times B$ on a $\sqrt{p} \times \sqrt{p}$ mesh of processors. Each processor holds one block of A and B . After an **initial skew** (row i of A shifted left by i , column j of B shifted up by j), the algorithm alternates **multiply-accumulate** with **cyclic shifts** of blocks. Uses only $O(1)$ block storage per processor with nearest-neighbour communication — memory-efficient and scalable versus a naive broadcast approach.

- **Decomposition:** 2D block (checkerboard) partition of the matrices.
- **Communication:** only left/up neighbour shifts on a torus — low, local, predictable.
- **Contrast:** Fox’s algorithm broadcasts A blocks along rows instead of skewing.

8 Distributed Coordination & Mutual Exclusion

⚠ Why It's Hard

In a distributed system there is **no shared memory** and **no global clock**. Mutexes and semaphores from Part I don't apply directly — mutual exclusion, election, and ordering must be built from **messages** alone, tolerating variable delays and failures.

8.1 Distributed Mutual Exclusion

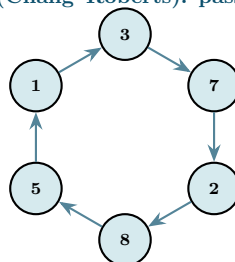
Algorithm	Mechanism	Msgs / entry	Weakness
Centralized	A coordinator grants a single token/lock on request	3	Coordinator is bottleneck/SPOF
Token Ring	A token circulates; holder may enter CS	1 to ∞	Token loss; latency
Ricart–Agrawala	Timestamped requests; enter after all others reply OK	$2(N-1)$	Every node must reply; failure-sensitive
Mackawa	Request permission from a voting quorum $\approx \sqrt{N}$	$\approx 3\sqrt{N}$	Deadlock possible; needs care

💡 Ricart–Agrawala (Lamport-clock based)

To enter the critical section, a process broadcasts a **timestamped REQUEST** to all others. A receiver replies **OK** immediately unless it is in (or wants) the CS with an *earlier* timestamp, in which case it **defers** the reply. A process enters once it has received OK from **all** $N-1$ peers. Ties broken by process id. Guarantees mutual exclusion and fairness using logical clocks.

8.2 Leader Election

Ring Election (Chang–Roberts): pass max id around



highest id (8) survives a full loop $\Rightarrow$ becomes leader

Algorithm	How the leader is chosen
Bully	A node that notices the leader is down sends ELECTION to all <i>higher</i> -id nodes; if none answer it declares itself leader (COORDINATOR); higher nodes “bully” lower ones. $O(N^2)$ msgs worst case.

Algorithm	How the leader is chosen
Chang–Roberts (Ring)	Each node forwards the maximum id it has seen around a logical ring; the node whose id returns to it is the leader. $O(N)–O(N^2)$ msgs.

8.3 Distributed Deadlock

- Detected via a global **Wait-For Graph (WFG)**; a cycle indicates deadlock.
- Techniques: centralized detection, edge-chasing (**Chandy–Misra–Haas** probe messages), and **timeouts**.
- Prevention via ordering / **wait–die** & **wound–wait** timestamp schemes (from distributed transactions).

9 Distributed Consensus & Agreement

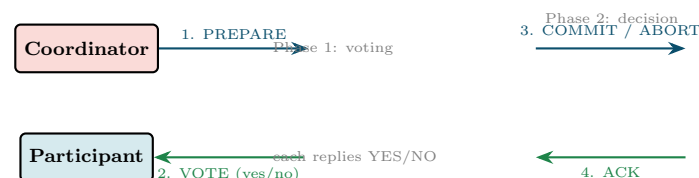
The Consensus Problem

A set of processes must **agree on a single value** despite failures, satisfying: **Agreement** (no two decide differently), **Validity** (the value was proposed), and **Termination** (every correct process eventually decides).

FLP Impossibility Result

In a **purely asynchronous** system, **no deterministic** consensus algorithm can guarantee termination if even **one** process may crash — you cannot distinguish a slow process from a dead one. Practical systems escape FLP using **timeouts** (partial synchrony), **randomization**, or **failure detectors**.

9.1 Atomic Commit: Two-Phase & Three-Phase Commit



- **2PC**: Phase 1 coordinator asks all to **PREPARE**; if *all* vote YES it broadcasts **COMMIT**, else **ABORT**. Guarantees atomicity but is **blocking**: if the coordinator crashes after PREPARE, participants holding locks can wait indefinitely.
- **3PC**: inserts a **pre-commit** phase so participants can reach a timeout-based decision without the coordinator, making it **non-blocking** under crash faults — but it fails under network partitions and is rarely used in practice.

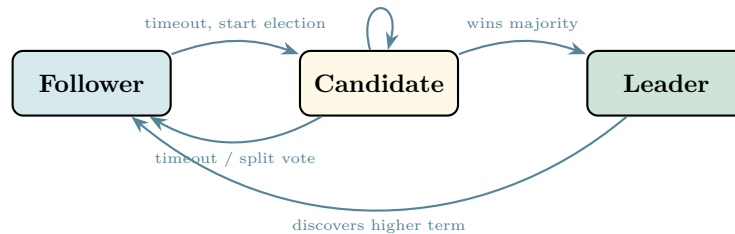
9.2 Paxos

Paxos Roles & Phases

Roles: **Proposers**, **Acceptors**, **Learners**. Two phases: (1) **Prepare/Promise** — a proposer picks a number n , asks a majority of acceptors to promise not to accept anything numbered $< n$; (2) **Accept/Accepted** — the proposer asks acceptors to accept value v with

number n . A value chosen by a **majority quorum** is final. Safe despite crashes/message loss; famously subtle. **Multi-Paxos** amortises phase 1 by electing a stable leader.

9.3 Raft — Understandable Consensus



- **Leader election:** time is divided into **terms**; a candidate that wins a majority vote becomes leader and sends periodic **heartbeats**.
- **Log replication:** clients talk only to the leader, which appends entries and replicates them; an entry is **committed** once stored on a majority, then applied to the state machine.
- **Safety:** a leader must have the most up-to-date log to be elected, guaranteeing committed entries are never lost. Equivalent power to Multi-Paxos, designed for **understandability**.

9.4 Byzantine Fault Tolerance (PBFT)

⚠ Byzantine Consensus

When up to f nodes may act **arbitrarily/maliciously** (send conflicting messages), consensus needs $n \geq 3f + 1$ nodes. **PBFT** (Practical BFT) reaches agreement in three phases — **pre-prepare** → **prepare** → **commit** — tolerating f Byzantine faults with $O(n^2)$ messaging. This underlies permissioned blockchains; public blockchains instead use **Proof-of-Work** / **Proof-of-Stake** (probabilistic Nakamoto consensus).

Protocol	Fault model		Quorum / bound	Property
2PC	Crash		All must vote	Atomic but blocking
3PC	Crash		Majority + timeout	Non-blocking; fails on partition
Paxos / Raft	Crash (non-Byz.)	(non-Byz.)	Majority ($> n/2$)	Safe, live under partial synchrony
PBFT	Byzantine		$n \geq 3f + 1$	Tolerates malicious nodes

10 Distributed Transactions & Data

📖 ACID vs BASE

ACID (traditional RDBMS): **A**tomicity, **C**onsistency, **I**solation, **D**urability — strong guarantees, harder to scale out.

BASE (many NoSQL): **B**asically **A**vailable, **S**oft state, **E**ventual consistency — trade strict consistency for availability and scale. This is the CAP trade-off in practice.

10.1 Concurrency Control & Quorums

- **Isolation** via two-phase locking (2PL), timestamp ordering, or **MVCC** (multi-version concurrency control — readers don't block writers).
- **Quorum consistency**: with N replicas, a write acknowledged by W and a read querying R replicas give **strong consistency** when $R + W > N$ (read and write sets overlap). Tuning R, W trades latency vs consistency (Dynamo-style).
- **Conflict resolution** for multi-master writes: last-writer-wins, version vectors, or CRDTs (conflict-free replicated data types).

10.2 Replication Strategies & NoSQL Models

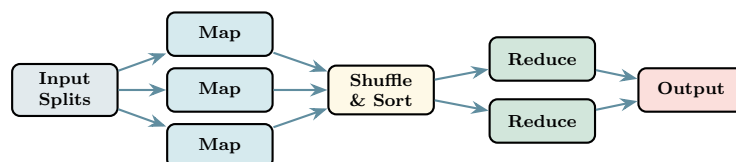
NoSQL type	Data model	Examples
Key-Value	Hash map of key $\rightarrow$ blob	Redis, DynamoDB, Riak
Document	JSON/BSON documents	MongoDB, CouchDB
Column-family	Sparse wide rows by column families	Cassandra, HBase
Graph	Nodes + edges + properties	Neo4j, JanusGraph

- **Primary-Backup (master-slave)**: one writer, replicas follow; simple, primary is a bottleneck.
- **Multi-Primary**: several writers; needs conflict resolution.
- **Sharding + Replication**: partition data across nodes (horizontal scale) *and* replicate each shard (fault tolerance) — the standard large-scale pattern.

11 MapReduce & Big-Data Frameworks

💡 The MapReduce Model (Google, 2004)

Express a computation as two functions: **Map**(k_1, v_1) $\rightarrow$ list(k_2, v_2) processes input in parallel; the framework **shuffles/sorts** by k_2 ; **Reduce**($k_2, \text{list}(v_2)$) $\rightarrow$ list(v_3) aggregates per key. The framework handles **parallelization, data distribution, fault tolerance, and scheduling** — the programmer writes only map & reduce.



🔗 Word Count — the “Hello World” of MapReduce (pseudocode)

```

def map(doc_name, text):
    for word in text.split():
        emit(word, 1)           # (word, 1) pairs

def reduce(word, counts):
    emit(word, sum(counts))     # total occurrences per word
  
```

11.1 Hadoop Ecosystem

- **HDFS** — distributed storage (see §13).
- **YARN** — cluster resource manager / job scheduler.
- **MapReduce engine** — batch execution on HDFS data (disk-heavy: writes intermediate results to disk between stages).
- Higher-level: **Hive** (SQL-on-Hadoop), **Pig**, **HBase** (column store).

11.2 Apache Spark

💡 RDDs and In-Memory Computing

Spark's core abstraction is the **RDD (Resilient Distributed Dataset)** — an immutable, partitioned collection computed in parallel. Spark keeps data **in memory** across operations, making iterative and interactive workloads far faster than disk-based MapReduce (often 10–100×).

- **Transformations** (`map`, `filter`, `flatMap`, `reduceByKey`, `join`) are **lazy** — they build a DAG (lineage) but don't execute.
- **Actions** (`collect`, `count`, `save`, `reduce`) trigger execution of the DAG.
- **Fault tolerance** via **lineage**: a lost partition is recomputed from its parent RDDs — no need to replicate all data.
- Libraries: **Spark SQL**/DataFrames, **MLlib**, **GraphX**, **Structured Streaming**.

🔗 Spark Word Count (PySpark)

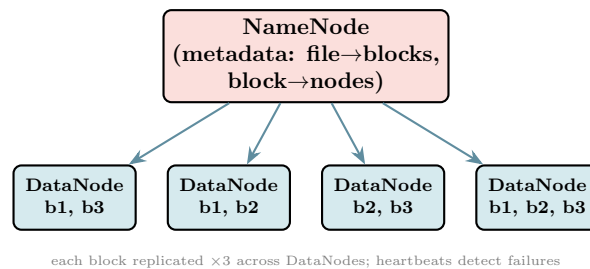
```
counts = (sc.textFile("hdfs://input.txt")
          .flatMap(lambda line: line.split())      # transformation
          (lazy)
          .map(lambda w: (w, 1))                  #
            transformation (lazy)
          .reduceByKey(lambda a, b: a + b))        #
          transformation (lazy)
counts.saveAsTextFile("hdfs://output")           # ACTION ->
# runs the DAG
```

Aspect	Hadoop MapReduce	Apache Spark
Data flow	Disk between stages	In-memory (RDD)
Speed	Slower (I/O bound)	10–100× for iterative jobs
Model	Map + Reduce only	Rich operators + DAG
Fault tolerance	Re-run tasks, replication	RDD lineage recomputation
Best for	Huge one-pass batch	Iterative, interactive, ML, streaming

12 Distributed File Systems & Storage

💡 GFS / HDFS Design Principles

Built for **commodity hardware** where failures are the norm. Files are split into large **blocks** (HDFS default 128 MB) spread across nodes and **replicated** (default 3×) for fault tolerance. Optimised for **large streaming reads** and **append/write-once**, not small random writes.



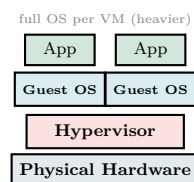
- **NameNode (master):** holds all metadata (namespace, block locations); a **single point of failure** historically — mitigated by a standby NameNode + journal.
- **DataNodes (workers):** store actual block replicas; send periodic **heartbeats** + block reports.
- **Rack awareness:** replicas placed across racks so a rack failure doesn't lose data; **re-replication** restores the replication factor after a node dies.
- **Object storage** (Amazon S3, Ceph): flat namespace of objects with rich metadata; the backbone of cloud data lakes.

13 Cloud, Virtualization & Containers

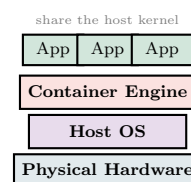
📁 Virtualization

Abstracting physical hardware so multiple isolated **virtual machines (VMs)** share one physical host, each with its own OS. Managed by a **hypervisor**: **Type 1** (bare-metal, e.g. ESXi, Xen, Hyper-V) runs directly on hardware; **Type 2** (hosted, e.g. VirtualBox, VMware Workstation) runs atop a host OS.

Virtual Machines



Containers



⚠️ VMs vs Containers

VMs virtualize *hardware* — each carries a full guest OS: strong isolation, heavier (GBs, slow boot). **Containers** (Docker) virtualize the *OS* — they share the host kernel and package just the app + dependencies: lightweight (MBs, sub-second start), less isolation. Containers made **microservices** and elastic scaling practical.

- **Docker:** build once (image), run anywhere; layered images, registries.
- **Kubernetes:** orchestrates containers across a cluster — scheduling, self-healing, service discovery, load balancing, auto-scaling (pods, deployments, services).
- **Serverless / FaaS** (AWS Lambda): run functions on demand; no server management, pay per invocation, auto-scales to zero.
- **Service models (recap):** **IaaS** (VMs/storage) → **PaaS** (managed runtime) → **SaaS** (ready apps); deployment models: public, private, hybrid, community.

💡 Cluster vs Grid vs Cloud

Cluster: tightly-coupled homogeneous nodes on a LAN, one administrative domain (HPC).

Grid: geographically dispersed, heterogeneous, multi-domain resource sharing. **Cloud:** on-demand, elastic, virtualized resources billed pay-as-you-go, hiding the infrastructure.

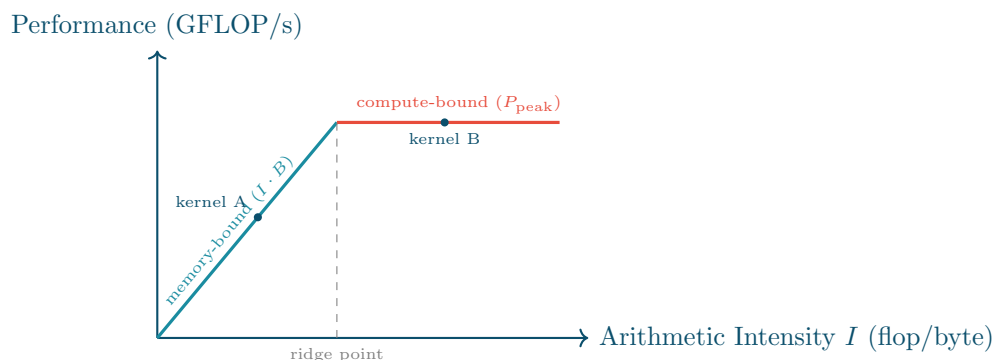
14 Performance Engineering & Analysis

📖 Arithmetic Intensity & the Roofline Model

Arithmetic (operational) intensity $I = \frac{\text{FLOPs}}{\text{bytes moved}}$ (flop/byte). The **Roofline model** bounds attainable performance:

$$P_{\text{attainable}} = \min(P_{\text{peak}}, I \times B_{\text{mem}})$$

where P_{peak} is peak compute (FLOP/s) and B_{mem} is peak memory bandwidth (byte/s). Low- I kernels are **memory-bound** (on the slanted roof); high- I kernels are **compute-bound** (under the flat roof).



14.1 Diagnosing and Improving Performance

- **Profile first:** tools like `gprof`, `perf`, Intel VTune, NVIDIA Nsight, Scalasca reveal hotspots; never optimise by guessing.
- **Overlap communication & computation:** use non-blocking MPI / CUDA streams so transfers hide behind useful work.
- **Improve locality:** blocking/tiling for cache, avoid false sharing, coalesce memory.
- **Reduce synchronization:** fewer barriers/locks; prefer reductions and lock-free structures.
- **Amdahl in practice:** attack the *largest serial fraction* first; a fast section already near its limit yields little.

⚠ Sources of Parallel Overhead

Speedup falls short of ideal because of: **communication** (latency + bandwidth), **synchronization** (idle waiting at barriers/locks), **load imbalance**, **redundant computation**, and **serial bottlenecks**. Communication cost model: $T_{\text{comm}} = t_s + \frac{m}{B}$ (startup latency t_s + message size m / bandwidth B).

15 Emerging & Applied Topics

- **Heterogeneous computing:** combine CPUs + GPUs + FPGAs/TPUs; match each task to the best accelerator. Basis of modern AI/HPC (e.g. GPU clusters training large models with data/model parallelism + Allreduce).
- **Edge & fog computing:** push computation toward data sources (IoT devices, gateways) to cut latency and bandwidth; cloud handles heavy aggregation.
- **Green / energy-aware computing:** performance-per-watt matters as much as raw speed; DVFS, consolidation, and efficient scheduling reduce energy.
- **HPC ↔ Cloud convergence:** elastic cloud HPC, containers on supercomputers, and serverless data pipelines blur the old boundaries.

Final-Term Formula & Fact Sheet

★ Key Formulae & Numbers

Performance:

- Speedup $S = T_s/T_p$; Efficiency $E = S/p$
- Amdahl: $S = \frac{1}{(1-f) + f/p}$, $S_{\max} = \frac{1}{1-f}$
- Gustafson: $S = fp + (1-f)$
- Roofline: $P = \min(P_{\text{peak}}, I \cdot B)$
- Comm.: $T_{\text{comm}} = t_s + m/B$
- Work/Span: $T_p \geq \max(W/p, D)$

Distributed rules of thumb:

- Quorum strong consistency: $R + W > N$
- Byzantine (PBFT): $n \geq 3f + 1$
- Crash consensus quorum: $> n/2$
- CAP: choose 2 of C, A, P
- Ricart–Agrawala: $2(N-1)$ msgs/entry
- Maekawa quorum size $\approx \sqrt{N}$
- HDFS block 128 MB, replication $\times 3$

≡ High-Yield Comparison Grid

Pair		Left	Right
OpenMP vs Pthreads		Directive, implicit, easy	Library, explicit, fine control
CPU vs GPU		Few latency cores	Thousands throughput cores
2PC vs 3PC		Blocking on coord. crash	Non-blocking, fails on partition
Paxos/Raft vs PBFT		Crash faults, majority	Byzantine, $n \geq 3f+1$
Hadoop vs Spark		Disk, map+reduce	In-memory RDD, DAG
VM vs Container		Full guest OS, heavy	Shares kernel, light
Amdahl vs Gustafson		Fixed size, strong scaling	Growing size, weak scaling

💡 Exam Strategy

Expect: (1) a **numeric** problem (speedup/efficiency/Amdahl/Gustafson or quorum); (2) a **compare-and-contrast** (from the grid above); (3) an **algorithm trace** (Ricart–Agrawala, 2PC, Raft election, MapReduce word count, Cannon’s shift); (4) a **define & explain** on GPU/CUDA, CAP, consensus, or containers. Always state **assumptions**, show **formula** → **substitution** → **result**, and label **diagrams**.